

Prompt:

8/17/2026, 10:09:11 PM

lanjut Tahap V

Response:

8/17/2026, 10:09:12 PM

Tahap V — Event-Driven Backtesting & Market Replay Engine

Sekarang kita masuk ke salah satu tahap **paling penting** dari seluruh arsitektur kita.

Kalau Tahap U menjawab:

"Bagaimana keputusan trading diterjemahkan menjadi order yang realistis?"

maka Tahap V menjawab:

"Bagaimana kita menghidupkan kembali pasar masa lalu sedemikian rupa sehingga strategi kita hanya mengetahui informasi yang memang tersedia pada saat itu?"

Ini yang akan menentukan apakah hasil backtest kita **layak dipercaya** atau hanya terlihat bagus karena simulator terlalu ideal.

V.1. Perubahan paradigma

Backtest sederhana biasanya seperti ini:

$$Data_t \rightarrow Signal_t \rightarrow Position_t \rightarrow Return_t$$

Masalahnya, proses tersebut sering secara implisit mengasumsikan:

- order langsung terisi,
- harga eksekusi sama dengan harga signal,
- tidak ada latency,
- tidak ada spread,
- tidak ada partial fill,
- tidak ada queue,

- tidak ada rejection,
- tidak ada order cancellation,
- semua data tersedia secara simultan.

Untuk sistem kita, itu **tidak boleh**.

Kita ingin:

$Data \rightarrow Event \rightarrow State \rightarrow Decision \rightarrow Order \rightarrow Exchange \rightarrow Fill \rightarrow Position \rightarrow PnL$

V.2. Event-driven architecture

Pasar kita representasikan sebagai sequence:

$$E = \{e_1, e_2, \dots, e_N\}$$

dengan:

$$t_1 < t_2 < \dots < t_N.$$

Setiap event:

$$e_i = (t_i, type_i, data_i).$$

Contohnya:

$$e_i = (10:00:01.023, MARKET_UPDATE, BTC).$$

Atau:

$$e_j = (10:00:01.051, ORDER_FILL, Order_{17}).$$

V.3. Mengapa event lebih baik daripada candle?

Misalkan candle 1 menit:

$$[100, 105, 99, 103].$$

Kita hanya mengetahui:

$$O = 100, H = 105, L = 99, C = 103.$$

Tetapi kita tidak mengetahui urutan sebenarnya.

Apakah:

$$100 \rightarrow 105 \rightarrow 99 \rightarrow 103$$

atau:

$$100 \rightarrow 99 \rightarrow 105 \rightarrow 103?$$

Untuk strategi intraday dengan:

- SL,
- TP,
- limit order,

perbedaan tersebut sangat besar.

V.4. Ambiguity pada candle

Misalkan kita punya:

$$Entry = 100,$$

$$SL = 98,$$

$$TP = 105.$$

Candle:

$$O = 100, \quad H = 106, \quad L = 97, \quad C = 103.$$

Dari OHLC saja kita tidak tahu apakah:

$$TP \rightarrow SL$$

atau:

$$SL \rightarrow TP.$$

Tetapi PnL bisa sangat berbeda.

Karena itu:

$OHLC$ alone $\nexists$ sufficient for realistic execution.

V.5. Market replay

Kita ingin melakukan:

$Replay(Data_{historical})$

seolah-olah waktu berjalan dari masa lalu ke masa depan.

Pada setiap event:

t_i ,

engine hanya boleh membaca:

F_{t_i}

yaitu information set yang tersedia sampai t_i .

V.6. Filtration

Dalam bahasa stochastic process, kita mendefinisikan filtration:

$$F_t = \sigma(X_\tau : \tau \leq t).$$

Strategi kita harus memenuhi:

$a_t \in F_t.$

Artinya keputusan trading pada waktu t hanya boleh bergantung pada informasi yang tersedia hingga t .

Ini adalah fondasi matematis untuk menghindari:

look-ahead bias.

V.7. Contoh look-ahead bias

Misalkan kita ingin memprediksi return:

$$r_{t+1}.$$

Tetapi secara tidak sengaja feature menggunakan:

$$Close_{t+1}.$$

Maka:

$$X_t = f(Close_t, Close_{t+1}).$$

Strategi memang akan terlihat sangat bagus.

Tetapi:

$$Close_{t+1} \notin F_t.$$

Jadi strategi tersebut **tidak kausal**.

V.8. Causality condition

Secara formal:

$$\boxed{Decision_t = \pi(F_t)}.$$

Bukan:

$$Decision_t = \pi(F_{t+k})$$

untuk:

$$k > 0.$$

V.9. Market state

Kita definisikan state:

$$S_t.$$

Misalnya:

$$S_t = \begin{bmatrix} P_t \\ V_t \\ Spread_t \\ Depth_t \\ Volatility_t \\ BTC_t \\ ETH_t \\ Gold_t \\ DXY_t \\ Position_t \\ Cash_t \\ Orders_t \end{bmatrix}.$$

Kemudian:

$$S_{t+1} = F(S_t, e_{t+1}).$$

V.10. Strategy sebagai policy

Strategy kita sekarang dapat ditulis:

$$a_t = \pi_{\theta}(S_t).$$

dengan:

$$a_t = \{target\ position, order\ type, quantity, price, SL, TP\}.$$

Parameter:

$$\theta$$

berasal dari seluruh model kita:

- factor model,

- alpha,
 - regime,
 - risk,
 - optimizer,
 - execution policy.
-

V.11. Event loop

Secara konseptual:

```
while historical_data remains:

    event = next_event()

    update_market_state(event)

    update_orders(event)

    process_fills(event)

    update_positions()

    update_risk()

    generate_signal()

    optimize_target_position()

    generate_execution_plan()

    submit_orders()

    record_state()
```

Tetapi urutan ini harus sangat hati-hati.

Kita tidak boleh menggunakan informasi dari event sebelum event tersebut diproses.

V.12. Urutan kausal

Pada timestamp t :

$$\boxed{MarketUpdate \rightarrow StateUpdate \rightarrow Signal \rightarrow Order}$$

Order tersebut baru dapat diproses pada:

$$t + \Delta t_{latency}.$$

Jadi:

$$\boxed{t_{decision} < t_{exchange} \leq t_{fill}}.$$

V.13. Decision timestamp

Setiap keputusan harus mempunyai:

$$t_{decision}.$$

Contoh:

$$10 : 15 : 00.000.$$

Kemudian:

$$t_{submit} = 10 : 15 : 00.003,$$

$$t_{exchange} = 10 : 15 : 00.010,$$

$$t_{fill} = 10 : 15 : 00.021.$$

Dengan demikian kita dapat menghitung:

$$L_{total} = t_{fill} - t_{decision}.$$

V.14. Order object

Saya menyarankan setiap order memiliki struktur:

$$O_i = \{id, symbol, side, type, quantity, limit_price, stop_price, timestamp, status\}.$$

Misalnya:

$$O_{17} = \{BTC, BUY, LIMIT, 0.01, 60000, \dots\}.$$

V.15. Order lifecycle

Order:

NEW

kemudian:

SUBMITTED

kemudian:

ACKNOWLEDGED.

Setelah itu:

PARTIALLY_FILLED

atau:

FILLED.

Atau:

CANCELLED.

Atau:

REJECTED.

V.16. Position state

Untuk setiap asset:

$$q_t.$$

Cash:

$$C_t.$$

Portfolio value:

$$V_t = C_t + q_t P_t.$$

Untuk multi-asset:

$$V_t = C_t + \sum_i q_{i,t} P_{i,t}.$$

V.17. Realized dan unrealized PnL

Unrealized:

$$PnL_t^{unrealized} = q_t(P_t - P_{entry}).$$

Realized:

$$PnL_t^{realized} = \sum_{closed\ trades} PnL_j.$$

Total:

$$PnL_t = PnL_t^{realized} + PnL_t^{unrealized}.$$

V.18. Cash accounting

Untuk buy:

$$C_{t+1} = C_t - qP_{exec} - Fee.$$

Untuk sell:

$$C_{t+1} = C_t + qP_{exec} - Fee.$$

Ini sederhana tetapi sangat penting.

Jangan pernah menghitung return hanya dari:

$$P_{close}.$$

V.19. Short selling

Untuk short:

$$q_t < 0.$$

Portfolio value tetap:

$$V_t = C_t + q_t P_t.$$

Jika:

$$P_t \downarrow,$$

maka:

$$q_t(P_t - P_{entry}) > 0.$$

V.20. Leverage

Gross exposure:

$$G_t = \sum_i |q_{i,t} P_{i,t}|.$$

Leverage:

$$L_t = \frac{G_t}{V_t}.$$

Constraint:

$$L_t \leq L_{max}.$$

V.21. Margin

Jika short atau leveraged position digunakan, simulator harus mempunyai:

$$MarginRequirement_t.$$

Jika:

$$Equity_t < MaintenanceMargin_t,$$

maka:

$$ForcedLiquidation.$$

Ini harus disimulasikan.

V.22. Stop-loss bukan sekadar angka

Misalkan:

$$SL = 95.$$

Kita tidak boleh otomatis mengatakan:

$$P_{low} < 95 \Rightarrow Fill = 95.$$

Realitanya:

$$P_{trigger} = 95$$

kemudian market order dikirim.

Harga fill dapat:

$$P_{fill} < 95.$$

Jadi:

$$SL_{trigger} \boxminus SL_{execution}.$$

V.23. Stop-loss slippage

Untuk long:

$$P_{SL,fill} = P_{SL} - \epsilon_{SL}.$$

Dengan:

$$\epsilon_{SL} \geq 0.$$

Pada crash:

$$\epsilon_{SL}$$

dapat sangat besar.

Ini sangat penting untuk crypto dan small-cap stocks.

V.24. Take-profit juga tidak sempurna

Jika TP menggunakan limit:

$$P_{TP}.$$

Maka:

$$P_{fill} = P_{TP}$$

hanya jika liquidity tersedia.

Jika menggunakan market:

$$P_{fill} \exists P_{TP}$$

karena slippage.

V.25. OCO logic

Position biasanya mempunyai:

SL

dan:

TP .

Keduanya harus dihubungkan sebagai:

$OCO = One\ Cancels\ Other.$

Jika TP filled:

$Cancel(SL).$

Jika SL filled:

$Cancel(TP).$

V.26. Kalau tidak menggunakan OCO

Simulator dapat melakukan hal absurd:

$TP\ filled$

kemudian beberapa event berikutnya:

$SL\ filled.$

Hasilnya posisi menjadi terbalik.

Ini adalah salah satu bug backtest yang sangat umum.

V.27. Trailing stop

Misalkan:

$$SL_t = P_t - d.$$

Tetapi SL hanya boleh bergerak satu arah untuk long:

$$SL_{t+1} \geq SL_t.$$

Jadi:

$$SL_{t+1} = \max(SL_t, P_{t+1} - d).$$

Untuk short:

$$SL_{t+1} = \min(SL_t, P_{t+1} + d).$$

V.28. Break-even stop

Misalkan entry:

$$P_e.$$

Setelah profit mencapai threshold:

$$R_t \geq R_{BE},$$

kita ubah:

$$SL = P_e.$$

Namun sebenarnya kita harus mempertimbangkan biaya.

Jika fee + expected exit cost:

$$C,$$

maka true break-even:

$$P_{BE} = P_e + C.$$

Untuk long.

V.29. Dynamic TP

Ini menghubungkan langsung dengan ide kita sebelumnya.

Misalkan:

$$TP_t = f(\text{volatility}, \alpha, \text{regime}, \text{liquidity}, \text{position}_{age}).$$

Jadi TP tidak harus fixed.

Misalnya:

$$TP_t = P_e + k_t \sigma_t.$$

Dengan:

$$k_t = f(\alpha_t, \text{regime}_t).$$

V.30. Tetapi jangan mengubah TP karena hindsight

Misalnya harga sudah naik 5%.

Tidak boleh kemudian:

"Karena sekarang terlihat kuat, TP kita naikkan."

Kita hanya boleh menaikkan TP jika:

$$Decision_t = f(F_t).$$

V.31. Market replay untuk cross-asset system

State kita harus mencakup:

$$\mathbf{X}_t = [BTC, ETH, Altcoin, Gold, DXY, SPX, VIX, Rates, \dots].$$

Tetapi setiap asset memiliki timestamp sendiri.

Kita menggunakan:

Event Queue

yang menggabungkan semua event berdasarkan timestamp.

V.32. Priority queue

Misalkan:

$$E = \{e_1, e_2, \dots, e_N\}.$$

Sort:

$$t(e_1) \leq t(e_2) \leq \dots.$$

Engine selalu mengambil:

$e^* = \arg \min_{e \in E} t(e).$

Kemudian memprosesnya.

Ini merupakan inti event-driven simulator.

V.33. Data hierarchy

Saya menyarankan data kita dibagi menjadi:

Level 0

Raw market data.

Tick

Level 1

Order book:

$L1/L2/L3$.

Level 2

Trades:

$TradePrint$.

Level 3

Bars:

$1s, 1m, 5m, \dots$

Level 4

Derived features:

$Volatility, Momentum, Correlation, Flow$.

Level 5

Model state:

$Alpha, Regime, Risk$.

V.34. Jangan mencampur level

Raw data:

D_{raw}

harus immutable.

Kemudian:

$D_{raw} \rightarrow D_{clean} \rightarrow D_{feature} \rightarrow D_{signal}$.

Ini memungkinkan kita melakukan audit.

V.35. Point-in-time dataset

Untuk setiap feature:

$$X_{i,t}$$

kita harus menyimpan:

$$timestamp_{observation}$$

dan:

$$timestamp_{availability}.$$

Karena:

$$timestamp_{observation} \sqsubseteq timestamp_{availability}.$$

V.36. Contoh fundamental data

Misalnya laporan perusahaan:

Q1.

Tanggal laporan:

May 10.

Tetapi periode laporan:

Jan – Mar.

Kita tidak boleh menggunakan data Q1 sebelum:

May 10.

Maka:

AvailabilityDate



adalah yang menentukan kapan informasi boleh masuk ke model.

V.37. Ini disebut point-in-time data

Untuk setiap observation:

$$X = (value, observation_time, availability_time).$$

Backtest menggunakan:

availability_time.

V.38. Survivorship bias

Misalkan pada 2026 kita melihat:

Top 100 stocks.

Kemudian backtest 2021 menggunakan daftar tersebut.

Itu salah.

Karena pada 2021 kita belum tahu perusahaan mana yang akan menjadi top 100 pada 2026.

Universe harus:

$Universe_t$

dan berubah sesuai waktu.

V.39. Delisted assets

Jika perusahaan delisting pada:

2024,

kita tidak boleh menghapus seluruh history-nya dari database.

Justru:

harus tetap ada.

Jika tidak:

SurvivorshipBias ↑ .

V.40. Corporate actions

Untuk saham:

- split,
- dividend,
- rights issue,
- merger,
- delisting.

Semua harus dipertimbangkan.

Misalnya:

2 : 1 split.

Harga:

$100 \rightarrow 50.$

Tetapi posisi:

1 share → 2 shares.

Simulator harus mengubah quantity dan price secara konsisten.

V.41. Crypto memiliki masalah berbeda

Untuk crypto:

- exchange listing,
- delisting,
- token migration,

- redenomination,
- perpetual funding,
- liquidation,
- maker/taker fee,
- funding rate.

Jadi engine kita harus modular.

V.42. Funding rate

Untuk perpetual futures:

$$PnL_{funding} = qP \times fundingRate.$$

Secara periodik:

$$C_{funding} = Notional \times f.$$

Maka total PnL:

$$PnL = PnL_{price} + PnL_{funding} - Fees.$$

V.43. Karena itu strategi crypto dan saham tidak boleh hard-coded

Kita sebaiknya mempunyai interface:

`MarketModel`

dengan implementasi:

`CryptoMarketModel,`

`EquityMarketModel,`

`FXMarketModel,`

`FuturesMarketModel.`

V.44. Tetapi strategy tetap sama

Strategy menerima:

$$State_t.$$

Kemudian:

$$Action_t = \pi(State_t).$$

Market model yang menentukan bagaimana action tersebut dieksekusi.

Ini membuat arsitektur:

$$\boxed{\textit{Strategy agnostic}}$$

terhadap asset class.

V.45. Deterministic replay

Ini sangat penting untuk penelitian.

Jika kita menjalankan:

$$\textit{Backtest}(A)$$

dua kali pada dataset yang sama, seharusnya:

$$\textit{Result}_1 = \textit{Result}_2.$$

Kecuali kita memang memasukkan stochastic process.

Untuk stochastic model kita gunakan:

$$\textit{seed}.$$

Sehingga:

$$\textit{seed} = s$$

memberikan:

reproducible simulation.

V.46. Monte Carlo execution

Setelah deterministic replay bekerja, baru kita masukkan uncertainty.

Misalnya slippage:

$$\epsilon \sim \text{Distribution}(\sigma, \text{liquidity}).$$

Kemudian:

$$P_{fill} = P_{model} + \epsilon.$$

Kita lakukan:

$$N$$

simulasi.

V.47. Distribution of outcomes

Kita mendapatkan:

$$\{P_{nL_1}, \dots, P_{nL_N}\}.$$

Kemudian:

$$E[P_{nL}],$$

$$\text{Var}(P_{nL}),$$

$$\text{VaR},$$

$$\text{CVaR}.$$

Jadi bukan hanya:

"strategymenghasilkan23%."

tetapi:

$$E[R] = 23\%, \quad CI_{95\%} = [..., ...].$$

V.48. Stress testing

Kita juga mengubah:

spread,

slippage,

latency,

volatility,

impact.

Misalnya:

Spread $\times$ 2.

Jika strategy masih profitable:

Robustness $\uparrow$.

Jika langsung hancur:

StrategyFragile.

V.49. Execution stress matrix

Kita bisa membangun:

$$Performance = f(Slippage, Spread, Latency, Impact).$$

Contoh:

Scenario	Slippage	Spread	Latency	Sharpe
Optimistic	0.5×	0.5×	0.5×	...
Base	1×	1×	1×	...
Conservative	2×	2×	2×	...
Stress	5×	3×	5×	...

Ini jauh lebih informatif daripada satu angka backtest.

V.50. Walk-forward testing

Kita tidak boleh:

$$Train_{2021-2025}$$

kemudian:

$$Test_{2021-2025}.$$

Sebaliknya:

$$Train \rightarrow Validation \rightarrow Test$$

berdasarkan waktu.

V.51. Rolling walk-forward

Misalnya:

$$Train : 2021 - 2022$$

$$Validation : 2023Q1$$

Test : 2023Q2.

Kemudian window bergeser:

Train : 2021Q2 – 2023Q1

Validation : 2023Q2

Test : 2023Q3.

Dan seterusnya.

V.52. Parameter stability

Misalkan optimizer menghasilkan:

$$\theta^*.$$

Kita tidak ingin:

$$\theta^*$$

berubah drastis setiap periode.

Definisikan:

$$\Delta\theta_t = \|\theta_t - \theta_{t-1}\|.$$

Jika:

$$\Delta\theta_t \gg 0,$$

bisa jadi model overfit terhadap regime tertentu.

V.53. Hyperparameter optimization

Ini harus dilakukan **hanya pada training/validation**.

Bukan test.

Jika:

T_{test}

digunakan untuk memilih parameter terbaik, maka:

T_{test}

bukan lagi out-of-sample.

V.54. Nested optimization

Arsitektur kita akhirnya akan mempunyai:

$Data \rightarrow Model \rightarrow Portfolio \rightarrow Execution$

dan masing-masing memiliki parameter.

Kita harus membedakan:

θ_{α} ,

$\theta_{portfolio}$,

$\theta_{execution}$.

V.55. Jangan optimalkan semuanya sekaligus

Jika kita langsung melakukan:

$$\arg \max_{\theta} Sharpe(\theta)$$

terhadap:

- α ,
- risk,
- QUBO,
- execution,

- TP,
- SL,
- trailing,
- leverage,

kita hampir pasti memperoleh overfitting.

Lebih baik:

Layered validation.

V.56. Validation hierarchy

Layer 1

Market data integrity.

Layer 2

Feature correctness.

Layer 3

Signal correctness.

Layer 4

Portfolio optimization.

Layer 5

Execution.

Layer 6

Full-system backtest.

Layer 7

Stress test.

Layer 8

Out-of-sample.

Layer 9

Paper trading.

Layer 10

Live trading dengan size sangat kecil.

V.57. Ini sangat penting untuk proyek kita

Karena kita menggunakan:

QUBO,

Ising,

VQE,

QAOA,

kita harus membuktikan bahwa improvement bukan berasal dari:

- look-ahead,
- data leakage,
- unrealistic execution,
- parameter tuning,
- random seed,
- simulator artifact.

V.58. Quantum benchmark

Misalkan classical optimizer menghasilkan:

R_C .

Quantum optimizer menghasilkan:

R_Q .

Kita tidak boleh langsung menyimpulkan:

$$R_Q > R_C \Rightarrow \text{Quantum advantage.}$$

Kita harus memastikan:

Data, Constraints, Initialization, Execution, Risk, TransactionCost

semuanya sama.

V.59. Paired experiment

Untuk setiap periode:

$$t,$$

jalankan:

$$\text{Strategy}_C(t)$$

dan:

$$\text{Strategy}_Q(t)$$

dengan kondisi identik.

Kemudian:

$$\Delta R_t = R_Q(t) - R_C(t).$$

Kita bisa menguji:

$$H_0 : E[\Delta R] = 0.$$

V.60. Dan bukan hanya return

Bandingkan:

$$\text{Sharpe,}$$

Sortino,

MaxDD,

CVaR,

Turnover,

ExecutionCost,

FillRate,

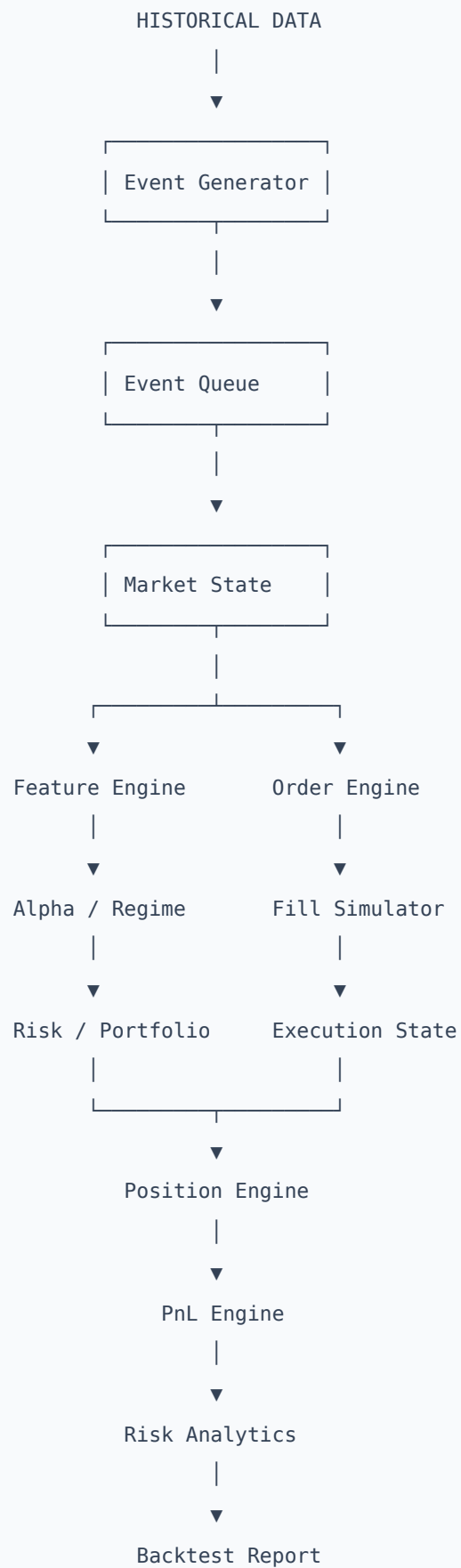
Latency,

PnL.

Jadi benchmark menjadi multidimensional.

V.61. Architecture final Tahap V

Secara keseluruhan:



V.62. Output dari engine

Setiap event sebaiknya menghasilkan log:

$$L_t = [\text{timestamp}, \text{state}, \text{signal}, \text{action}, \text{order}, \text{fill}, \text{position}, PnL, \text{risk}].$$

Dengan ini kita dapat melakukan **full forensic analysis** terhadap setiap trade.

V.63. Trade journal otomatis

Untuk setiap trade:

$$Trade_i = \{\text{signal}, \text{entry}, \text{execution}, \text{exit}, PnL, \text{risk}, \text{slippage}, MAE, MFE, \text{holding time}\}.$$

Dua metrik yang sangat penting:

MAE

Maximum Adverse Excursion:

$$MAE_i = \min_t(R_{i,t}).$$

MFE

Maximum Favorable Excursion:

$$MFE_i = \max_t(R_{i,t}).$$

V.64. Mengapa MAE/MFE penting?

Misalnya kita menemukan:

$$MAE$$

sebagian besar trade hanya:

$$-0.3R,$$

tetapi SL kita:

$$-1R.$$

Mungkin SL terlalu longgar.

Sebaliknya jika:

$$MAE \approx -0.9R$$

sebelum trade yang akhirnya menang,

SL terlalu ketat dapat membunuh strategy.

Jadi:

$$MAE/MFE \rightarrow SL/TP \text{ design.}$$

V.65. Feedback loop

Sekarang sistem kita menjadi benar-benar adaptif:

$$Backtest \rightarrow Diagnostics \rightarrow ModelImprovement \rightarrow Backtest$$

Tetapi hati-hati:

$$Test$$

tidak boleh masuk ke feedback loop.

Kalau masuk:

$$Test \rightarrow Optimization$$

maka:

$$Test$$

sudah terkontaminasi.

V.66. Tiga dataset

Saya ingin kita menetapkan tiga wilayah:

TRAIN

untuk membangun model.

VALIDATION

untuk memilih model/parameter.

TEST

untuk evaluasi final.

Kemudian setelah test:

PAPER

dan baru:

LIVE.

V.67. Dan ada satu lapisan lagi: Shadow Mode

Sebelum live execution:

Market → Strategy

tetapi:

Order ⇌ Sent.

Kita hanya mencatat:

$\hat{P}_{fill},$

$\hat{P}_{nL},$

$\hat{Slippage}.$

Kemudian dibandingkan dengan market sebenarnya.

Ini disebut:

Shadow Trading.

V.68. Hierarki deployment kita

Saya sarankan:

Backtest → StressTest → WalkForward → MonteCarlo → Shadow → Paper → MicroLive → Scale.

Bukan:

Backtest → Real Money.

V.69. Apa yang sebenarnya kita bangun?

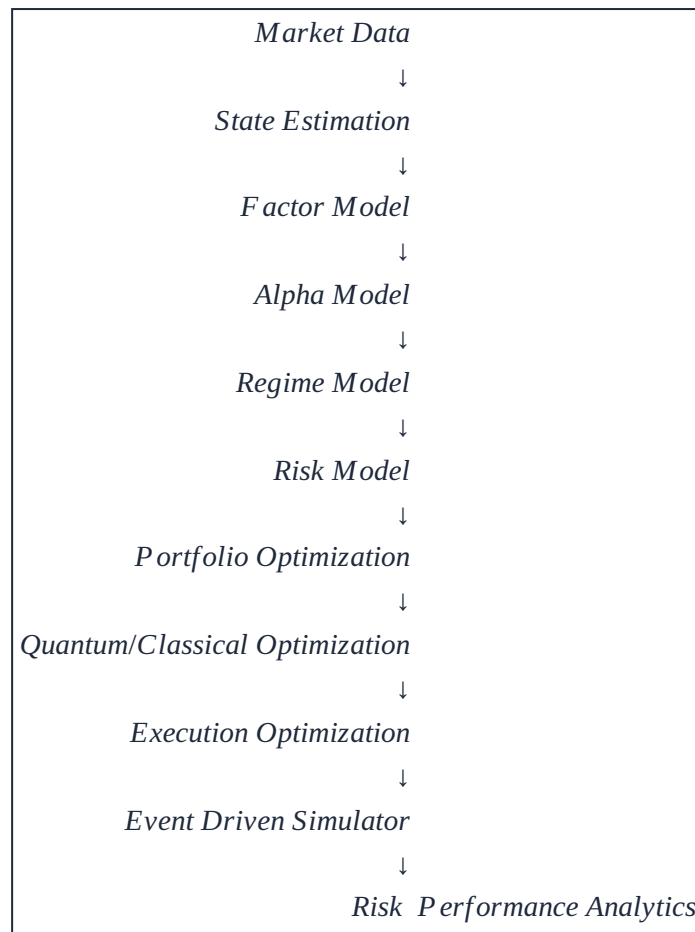
Pada titik ini, proyek kita sudah jauh melampaui:

"bot trading."

Kita sedang membangun:

Research-grade Quantitative Trading Platform

dengan struktur:



V.70. Tahap V dalam satu persamaan

Kalau seluruh Tahap V harus diringkas dalam satu persamaan, saya akan menuliskannya sebagai:

$$S_{t+1} = F(S_t, E_{t+1}, \pi_\theta(F_t))$$

dengan constraint:

$$\pi_\theta(F_t) \in A_t$$

dan PnL:

$$W_{t+1} = W_t + PnL(S_t, E_{t+1}, a_t) - C_{execution}.$$

Karena:

$$a_t \in F_t,$$

maka sistem bersifat **causal**.

Dan karena:

$$E_{t+1}$$

diproses satu per satu, kita memperoleh:

event-driven market replay.

V.71. Hubungan Tahap A → V

Sekarang kita dapat melihat mengapa tahap-tahap sebelumnya sebenarnya saling mengunci:

$$A \rightarrow B \rightarrow C \rightarrow \cdots \rightarrow U \rightarrow V$$

bukan kumpulan modul terpisah.

Secara konseptual:

Market → Information → Prediction → Optimization → Risk → Execution → Reality Check.

Tahap V adalah **reality check** tersebut.

V.72. Hal yang harus kita implementasikan setelah ini

Sebelum masuk ke tahap berikutnya, saya justru menyarankan kita **tidak langsung melanjutkan menambah teori**.

Kita sudah mempunyai cukup banyak teori untuk mulai membangun engine pertama.

Urutan implementasi yang paling masuk akal:

V.1 — Data schema

Bangun format:

```
MarketEvent
TradeEvent
OrderBookEvent
OrderEvent
FillEvent
PositionEvent
```

V.2 — Event queue

Implementasikan:

PriorityQueue(E)

berdasarkan timestamp.

V.3 — Market state

Implementasikan:

S_t .

V.4 — Order state machine

Implementasikan:

NEW → SUBMITTED → FILLED/CANCELLED/REJECTED.

V.5 — Execution simulator

Mulai dari:

MarketOrder

kemudian:

LimitOrder

kemudian:

PartialFill.

V.6 — Position & PnL

Implementasikan accounting yang benar.

V.7 — SL/TP/OCO

Kemudian:

Trailing/BEP/DynamicTP.

V.8 — Transaction cost

Masukkan:

fee + spread + slippage + impact.

V.9 — Walk-forward engine

Baru setelah simulator benar:

Train → Validate → Test.

V.10 — Monte Carlo & stress

Dan terakhir:

Robustness.

V.73. Satu keputusan desain yang sangat penting

Untuk proyek kita, saya menyarankan **jangan membuat backtest engine yang hanya menerima dataframe OHLC lalu menghasilkan equity curve.**

Kita sebaiknya membangun:

Market Replay Engine

yang menerima event:

$E_1, E_2, \dots, E_N$

dan menghasilkan:

$$S_1, S_2, \dots, S_N.$$

Dengan begitu, engine yang sama nantinya dapat digunakan untuk:

Historical Backtest

dan kemudian, dengan interface yang sama:

Paper Trading

dan bahkan:

Live Trading.

Artinya kita tidak perlu memiliki tiga sistem berbeda. Kita memiliki **satu trading engine**, hanya sumber event-nya yang berubah:



Menurut saya, ini adalah titik arsitektural yang sangat penting untuk proyek kita: **backtest bukan versi mainan dari live trading; backtest adalah live engine yang diberi historical event stream.**

Setelah fondasi ini selesai, **Tahap W** secara alami dapat kita gunakan untuk membangun **Performance Attribution, Risk Analytics, dan Robustness Framework**—yaitu menjawab *mengapa* sistem menghasilkan profit/loss, bukan hanya *berapa* profit/loss-nya.